A dark software factory built in your own codespace.

Rasa Mantle and the OpenAI Agents API. Nothing to install.

A request.
A builder.
A check it cannot edit.

Build a small helper.
Let an agent propose a change.
Decide what is released.

Three hours, mostly your hands on the keys.

Tuesday, November 3, 2026, 3:00 to 6:00 PM New York time.

Result	Helper	Reset handling	Source check	Break	Builder	Gate	Release	Preview
10	20	20	20	5	40	25	20	20

Block lengths in minutes. Break at 4:10 PM. The builder starts at 4:15 PM. Close at 6:00 PM.

Open your codespace now, at minute 0.

You bring two keys and a personal GitHub account. Nothing is set up in advance.

1 · Account	A personal GitHub account, not one managed by an enterprise
2 · Keys	Your free Rasa Developer Edition key and your OpenAI key
3 · Secrets	`RASA_LICENSE` and `OPENAI_API_KEY`, scoped to the repository you open
4 · Create	New with options, the `software-factories-workshop` configuration, at least 4 cores

Added a secret to a codespace that was already running? Stop it, start it again, run the setup script again, and wait for the line that starts Setup finished. Steps: ATTENDEE-PACK.md.

One small agent: a router restart helper.

A customer says “restart my router”. The command is simulated.

You type	You should see
Restart my router	It asks you to confirm
Yes	It records one command, `REBOOT`
Reset it	It asks whether you mean a restart or a factory reset, and sends nothing
Factory reset	It says it cannot do that here and offers a person

Minutes 10 to 30: build your helper.

The steps in short. The exact commands are in the attendee pack.

```
mkdir ~/router-helper-agent && cd ~/router-helper-agent
rasa init --engine mantle --no-prompt
python install_helper.py .
rasa train
rasa inspect --port 5010
```

A summary, not for copying: the real commands use full paths into the codespace. Copy them from ATTENDEE-PACK.md.

Target at minute 30: all four rows.

Compare what you see with the table. The meaning must match.

What to aim for

A confirmation, then `REBOOT`.
A question when you say reset.
A refusal, with an offer of a person.

Not there by minute 25?

Raise your hand.
The presenter helps the room catch up at minute 30.

Minutes 30 to 50: try to break your helper.

Use your own words. Change your mind. Ask for the wrong thing.

01

Change your mind

Ask for a restart, then say you do not want it.

02

Ask in other words

Ask for a reset three different ways.

03

Ask for the dangerous one

Ask for a factory reset outright.

A check that reads the file, not the reply.

The helper must never record `FACTORY_RESET` for a restart.

```
run check_helper.py on tools.py
  PASS: a restart records REBOOT
change "REBOOT" to "FACTORY_RESET", run it again
  FAIL
change it back to "REBOOT", run it again
  PASS
```

A summary, not for copying: the exact command is in ATTENDEE-PACK.md. A pass reads the configured command. It does not prove the helper ran it in a live conversation.

Break.

Five minutes. Back at 4:15 PM.

Back at 4:15 PM.

Leave your codespace open. If your four conversation checks pass, and the source-file check passes again after you restore REBOOT, you have a working helper and a check for it.

The factory begins: your builder.

An agent made with the OpenAI Agents API writes one small change.

Task	Add one read-only operation, “check line status”
Your key needs	`api.agents.read`, `api.agents.write`, `api.responses.write`
One run	A few minutes and a paid model call on your own key
It ends with	`saved: True`, the file’s absolute path, and its sha256

Put the builder behind your helper.

A small tool server hands your request to the builder.

Terminal 1	Runs the builder once. It ends by itself.
Terminal 2	The tool server. Leave it running until the end.
Terminal 3	Stop the Inspector, install the skill, train, open the Inspector again.

Your message goes to the Mantle helper, to the tool server, to the builder.

A request goes in. A proposed change comes out.

Ask the helper: `add a check that tells me the line status.`

Expected: one message

Starts with `Build` and a build id.
Says it saved a file, with an absolute path ending in `check_line_status.py`.
Shows a sha256.
Ends: Nothing was applied to the helper.

Read the other replies

`No build started`: another build was running.
`I cannot tell whether a build started`: look at terminal 2 before asking again.

A gate the builder cannot edit.

It judges the saved file and never runs it.

File	Verdict
Your builder’s saved file	The gate decides
`good.py`	ACCEPT
`bad_opens_network.py`	REJECT
`bad_sends_reset.py`	REJECT
`bad_writes_a_file.py`	REJECT

One function, `check_line_status()`, plain values. Not a general sandbox.

You are the last gate.

Record approve or refuse, and why. Nothing is installed or applied.

01

Ask

Request the change through the helper.

02

Judge

The builder proposes. The gate says ACCEPT or REJECT.

03

Decide

You record approve or refuse, for manual review.

A Studio 2 preview.

Shown from the presenter’s own machine. You cannot try it yet.

An early preview

Nothing in this workshop needs it.
You do not need to do anything.

Hear when Studio is released

Join the Rasa Community:
`info.rasa.com/community`

Describe, build, check, release.

The check belongs to someone other than the builder.

“What evidence would let us release this change without reading every line?”

Start with one bounded change. Keep the rejected cases.